5. 拟采用的数据结构与算法

数据定义

拟定义 TeachingClass（教学班）、Meeting（上课时间）和 Schedule（合法组合方案）等数据结构对象，其中：

TeachingClass：表示一个可独立选择的课程候选项，记录所属课程、课程代号、任课教师、教室及全部上课时段。根据这些信息我们能唯一确定一个教学班。其中的全部上课时段对象是一个 Meeting 类型的 Vector。

Meeting：表示教学班的一段具体授课时间，包括周次、星期及起止节次，一个教学班可包含多个上课时段。

Schedule：表示由若干教学班构成的完整选课方案，需满足课程选择、时间冲突和用户硬约束，并保存早课、空堂、上课天数等评价指标。

主要数据结构

主要数据结构如下：

数据结构	存储对象与操作
顺序表与散列表	顺序表存储 TeachingClass、Meeting 及生成的 Schedule；散列表建立教学班编号索引及“课程标识—候选教学班集合”映射。支持教学班及授课记录的创建、插入、查找、修改、删除和清空。
时间位集合	将整个学期的“周次-星期-节次”编码为二进制占用位。每个 TeachingClass 的多个 Meeting 合并为一个时间位集合；通过集合交判断时间冲突，通过集合并维护当前组合的总时间占用。
无向图邻接表	以 TeachingClass 为顶点，当两个不同课程的教学班存在实际时间冲突时建立无向边，用于记录和查询教学班之间的冲突关系。
候选集合与回溯记录	保存搜索过程中每门课程当前仍可选择的教学班，以及某一搜索层因冲突而临时排除的候选项，便于回溯时准确恢复状态。
二叉堆	当合法组合数量较多时，仅保留按照用户偏好评价后较优的前 K 个 Schedule，使当前较差方案能够优先被替换，减少大量方案同时驻留内存。

拟采用算法

1) 冲突图模型

记教学班 a 在整个学期内占用的所有时间（“周次-星期-节次”结构）记为时间集合$T(a)$。若两个教学班 a 和 b 满足

$$T(\boldsymbol{a}) \cap \boldsymbol{T}(\boldsymbol{b}) \neq \emptyset$$

则认为二者存在实际时间冲突，并取交集中的占用位置反向推导出具体冲突周次、星期和节次，用于制作界面提示（为什么冲突）。

对于用户偏好设置的禁用时间，记其时间集合为F。教学班 a 能够被选择的必要不充分条件之一为

$$T(a) \cap F = \emptyset$$

对于当前已经选择的若干教学班，其总时间占用集合记为T_s。加入新的教学班 y 前，仅需判断

$$T(a) \cap T_s = \emptyset$$

若不存在交集，则将其加入当前方案，并更新T_s

$$T_s \leftarrow T_s \cup T(a)$$

在该模型中，每个 TeachingClass 为顶点，若两个不同课程的教学班，则在二者之间建立无向边，由此构建教学班冲突图G

$$G = (V, E)$$

其中根据课程代号（课程名称）将教学班划分为若干候选集合$V_1, V_2, \ldots, V_m$。一个合法的选课组合$S \subseteq V$需要同时满足

$$|S \cap V_i| = 1, \qquad i = 1,2,\ldots,m$$

即每门课程（同名课程）恰好选择一个教学班，并且任意两个已选择额的教学班之间均不存在冲突边。由此将课程选择要求、集合约束问题和图冲突统一起来。

冲突图的构建：

在全部 TeachingClass 的时间位集合建立完成后，对不同课程的教学班进行两两比较。若两个教学班的时间集合交集非空，则在冲突图中加入一条无向边。

邻接表用于保存每个教学班与哪些候选教学班发生冲突。组合搜索过程中选择某个教学班后，可直接访问其邻接顶点，快速确定需要排除的候选项。

教学班发生增加、删除或上课时间修改后，只重新计算与该教学班有关的时间集合和冲突边；已经生成且受到数据变动影响的旧 Schedule 标记为失效，避免继续展示过期结果。

2) 时间建模与冲突检测方法

为统一处理不同周次、同一教学班多次授课的情况，我们提出授课时间单元的概念：一个授课时间单元将整个学期离散化为有限的时间集合。设学期有W周、每周 7 天、每天 6 个节次，则总授课时间单元数量为

$$T = W \times 7 \times 6$$

将具体的一节课（第 week 周、星期 day、第 period 节）与位集合下标进行映射

$$index = (week - 1) \times 6 \times 7 + (day - 1) \times 6 + (period - 1)$$

对于某一特定 TeachingClass，将其全部 Meeting 对应的授课时间单元位置置 1，即得到该教学班完整的时间位集合。

我们提出一种基于授课时间单元的冲突检测方法：两个教学班之间的冲突检测可转化为两个位集合的按位与操作。若结果非零，则存在冲突；若结果为空，则二者时间兼容。

这种方法能够正确区分各种复杂的课况。

3) DFS 与回溯剪枝

若第i门课程共有m_i个教学班，在不考虑任何冲突的情况下，所有教学班组合数量最多为

$$\prod_{i=1}^{n} m_i$$

由此可见，随着课程和教学班数量增加，直接枚举的开销迅速增大。我们拟采用 DFS 与回溯算法逐门选择教学班。

每新增一个候选教学班，立即检查：

- 是否与已有固定课表冲突
- 是否与用户偏好（如禁用时段、锁定教学班等）冲突
- 是否与当前（固定课表上的）已选择教学班冲突

若任一条件不满足，立即终止当前分支搜索并进行回溯，不再探索后续组合。

在普通回溯基础上，我们拟提出两条加速优化策略：

- 最少剩余候选优先
- 候选提前筛除

其中：

最少剩余候选优先受贪心算法启发：搜索过程中不固定按照课程录入顺序选择，而是优先处理当前剩余合法教学班数量最少的课程，使约束较强的课程更早被确定，从而尽早发现无解分支。

候选提前筛除则是：每选择一个教学班后，根据冲突图临时排除后续课程中与其冲突的候选项；若某门必选课程已无可用教学班，则立即终止当前分支并回溯。

4) 个人偏好设置与基于大根堆的 Top-K 最优方案维护

只有满足所有要求的 Schedule 才允许进入评价阶段。系统拟根据整个学期的实际授课安排计算若干时间指标，拟包括：

- 第一大节（也称早八）节数
- 一周或整个学期的上课天数
- 单日最多/最少课程量
- 最大连续上课节数等

用户可设置不同指标的比较优先级，系统按照这些指标对合法方案进行排序。当合法方案数量较大时，使用二叉堆维护当前较优的前K个方案，以降低结果存储的空间开销。

工作量（阐述所选任务的工作量体现，包括具体的设计模块及预期的代码量等）

本课设虽然面向学生日常选课场景，但其实现过程需要同时完成数据建模、时间编码、冲突图构建、组合搜索、结果评价与 Qt 图形界面开发等多个模块，属于“数据结构 + 算法 +GUI”结合较紧密的课程设计项目。系统既要支持教学班、授课时段等基础数据的录入、修改、保存与恢复，又要在此基础上完成时间冲突检测、冲突关系维护、合法组合搜索、方案排序与结果展示，因此整体工作量较为充实。

拟初步将系统分为：

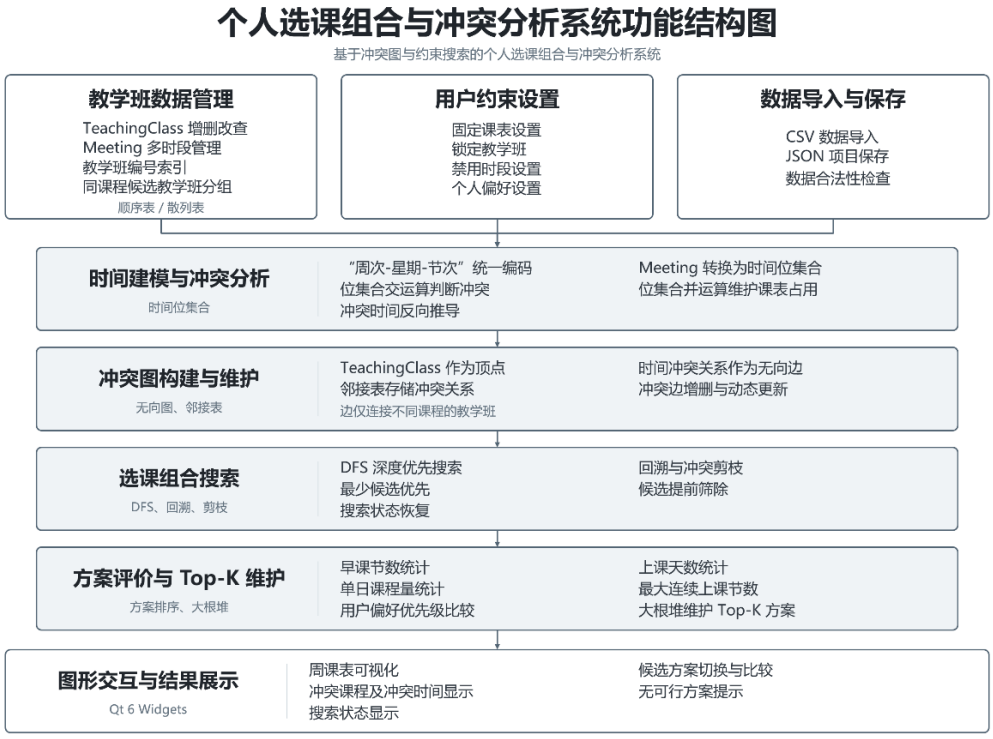
- 数据侧

- 算法侧
- 界面侧

拟实现系统的功能结构如下：

1) 教学班数据管理。包括：TeachingClass 的增删改查、Meeting 多时段录入与编辑、教学班编号索引与相同课程分组、CSV 导入处理等。
2) 时间与冲突分析。包括：“周次-星期-节次”结构时间编码、TeachingClass 时间位集合构建、固定课表/禁用时段冲突检测、TeachingClass 冲突检测、冲突时间反向推导与提示信息等。
3) 冲突图构建与维护。包括：无向图的邻接表构建、边（冲突边）增加、删除与更新、邻接教学班查询等。
4) 组合搜索与剪枝。包括：DFS/回溯选择教学班、冲突即时剪枝、最少候选优先、候选提前筛除、搜索中止与状态恢复等。
5) 个人偏好的方案评价与排序。包括：评价维度与指标设计、评价指标相关数据统计、用户偏好优先级排序、基于大根堆的 Top-K 方案维护
6) 用户图形界面与结果显示。包括：主界面与课程录入界面、个性化偏好设置（约束）界面、周课表展示、候选方案列表展示、冲突警示、无解提示与搜索状态说明等。

功能结构图如下：



预计整体代码量约为 3000-5000 行左右。

工作计划安排（要明确中期检查及软件验收时的完成目标）

1. 第一阶段（第 1-3 天）环境构建、需求细化与底层数据建模
   完成目标：
   - ◆ 明确输入输出格式，统一 TeachingClass、Meeting 和 Schedule 的数据代码定义；
   - ◆ 搭建 C++、Qt 6 和 CMake 环境
   - ◆ 完成 TeachingClass、Meeting 的基础存储结构设计；
   - ◆ 实现教学班顺序表存储、编号索引和课程分组索引；
   - ◆ 初步实现时间位集合的数据结构。
2. 第二阶段（第 4-7 天）时间建模与冲突检测
   - ◆ 实现“周次-星期-节次”到集合下标的映射；
   - ◆ 完成 TeachingClass 的时间位集合构建；
   - ◆ 实现固定课表、禁用时段与教学班之间的冲突检测；
   - ◆ 初步构建冲突图邻接表；
   - ◆ 初步搭建 GUI 界面，支持手工录入教学班及其授课时段。

   中期检查完成目标：

   系统应能够录入教学班数据，生成时间位集合，正确判断两个教学班是否冲突，并在界面上展示课程列表和初步冲突信息。
3. 第三阶段（第 8-11 天）组合搜索与剪枝优化
   - ◆ 实现基于 DFS 的回溯选班算法；
   - ◆ 实现冲突即时剪枝；
   - ◆ 实现“最少候选优先”策略；
   - ◆ 实现“候选提前筛除”及回溯恢复；
   - ◆ 能够在固定课表和用户硬约束下生成合法 Schedule。
4. 第四阶段（第 12-14 天）个性化方案评价、Top-K 维护与界面调试
   - ◆ 实现早课、空堂、上课天数、最大连续上课节数等评价指标；
   - ◆ 实现按用户优先级比较的方案排序；
   - ◆ 实现基于堆的 Top-K 候选方案维护；
   - ◆ 完成周课表显示、候选方案切换、冲突原因显示等 GUI 功能；
   - ◆ 实现 CSV 导入和 JSON 保存/恢复；
   - ◆ 进行模块联调和功能性测试。

软件验收完成目标：

- 教学班及授课时段录入；
- 固定课表和禁用时段设置；
- 时间冲突检测；
- 合法选课组合生成；
- 多方案评价与展示；
- 项目保存。

即完成“**录入—分析—搜索—展示**”的完整闭环。

同组设计者及分工（若自行完成则写“独立完成”；多人同组则要分工明确，清晰列出每人所承担的具体任务安排）

1. 组长：Me

负责系统总体架构以及较复杂的数据结构和组合搜索算法，主要工作如下：

1. 设计系统整体架构及数据层、算法层、界面层之间的接口；
2. 设计并实现基于 64 位整数数组的时间位集合，完成集合交、并及快速冲突判断；
3. 设计并实现无向冲突图邻接表，完成冲突图的构建、边更新及邻接关系查询；
4. 实现基于 DFS 的回溯选班算法；
5. 实现冲突即时剪枝、**最少候选优先**及**候选提前筛除**等搜索优化；
6. 实现候选集合的临时修改及回溯状态恢复；
7. 实现基于大根堆的 Top-K 候选方案维护算法；
8. 对普通枚举、普通回溯与优化回溯进行性能测试，统计搜索节点数和运行时间；
9. 负责核心模块的最终集成和系统联调；
10. 若基础功能提前完成，负责“尽量少改动原课表”的替代方案拓展。

主要数据结构：

- 时间位集合；
- 无向图邻接表；
- 候选集合；
- 回溯记录；
- 大根堆。

主要算法：

- 位集合交并运算；
- 冲突图构建与动态更新；
- DFS /回溯搜索；
- 冲突剪枝；
- 最少候选优先；

- 候选提前筛除；
- Top-K 堆维护。

2. 组员：L

负责教学班基础数据管理、部分冲突与评价算法、文件读写以及图形界面实现，主要工作如下：

1. 设计并实现 TeachingClass、Meeting 等基础数据的增删改查；
2. 使用顺序表和散列表维护教学班数据、教学班编号索引以及“课程代号—候选教学班”分组；
3. 实现 Meeting 数据的合法性检查与授课时段标准化处理；
4. 实现“周次—星期—节次”与授课时间单元编号之间的编码与反向推导算法；
5. 实现不依赖冲突图的逐条 Meeting 时间冲突检测算法，作为位集合冲突检测的独立验证方法；
6. 实现小规模数据下的完整组合枚举算法，用于与优化回溯算法的结果进行正确性核对；
7. 实现 Schedule 的方案评价算法，统计早课节数、上课天数、单日课程量、最大连续上课节数等指标；
8. 实现基于用户偏好优先级的多指标比较与方案排序算法；
9. 负责 CSV 数据导入；
10. 负责 Qt 主界面、教学班编辑、约束设置、周课表和候选方案列表等 GUI；
11. 实现冲突结果的可视化，将冲突时间解析成“第几周-星期几-第几节”的可读形式；
12. 负责基础功能测试、GUI 回归测试并协助准备验收数据。

主要数据结构：

- 顺序表；
- 散列表；
- Meeting 记录表；
- Schedule 顺序表；
- 课程候选分组集合。

主要算法：

- 散列表索引与查找；
- 授课时间编码与反向解析；
- 基于周次、星期和节次的直接时间重叠判断；
- 完整组合枚举；
- Schedule 指标统计；
- 多指标比较与排序；
- 数据合法性检查。